

CRYPTO READING GROUP

QUANTUM CRYPTOGRAPHY: QKD & QROM

Fall 2026

UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Contents

About **4**
- Quantum Cryptography . . . 4
- Format . . . 4
- Useful Information . . . 4
- Organizers . . . 4

Schedule **5**
- Fall 2026: QKD and QROM . . . 5

Plan **6**

Week 1: Quantum Computing **9**
- Classical Computing . . . 9
  - Complexity Classes . . . 10
- Introduction to Quantum . . . 10
- Quantum Computing . . . 15

Week 2: Quantum Information **16**

Week 3: Quantum Circuits **17**
- Classical Circuits . . . 17
- 1-Qubit Circuits . . . 18
  - Unitary Transformations . . . 19
- n-Qubit Circuits . . . 21
  - 2-Qubit Circuits . . . 22
  - 3-Qubit Circuits . . . 25
  - n-Qubit Circuits for $n \geq 4$ . . . 27
- Famous Quantum Circuits . . . 29
  - Deutsch's Algorithm . . . 30
  - Grover's Algorithm . . . 31
  - Quantum Fourier Transform . . . 35
  - Shor's Algorithm . . . 38

Week 4: Quantum Key Distribution **42**

Week 5: QKD Security **43**

Week 6: Deploying QKD **44**

Week 7: Classical Random Oracle Model **45**

Week 8: Quantum Random Oracle Model **46**

Week 9: QROM Lemmas **47**

Week 10: QROM Security Proof **48**

References **49**

About

This document contains the plan and notes for the Fall 2026 quantum cryptography reading group.

Quantum Cryptography

This semester, the reading group focuses on *quantum cryptography*. We will build the background in quantum computing and quantum information needed to study its protocols and security proofs. The group is an opportunity to learn these topics together, including material that is new to many of us.

In the first half of the semester, we will study quantum key distribution (QKD), its security model, and the challenges of deploying it. The second half turns to the random-oracle model (ROM), the quantum random-oracle model (QROM), and a worked example of a QROM security proof.

Format

Each session will be prepared by the speaker or speakers listed in the schedule. The goal is to encourage collaboration and leave room for discussion during the presentation.

When two speakers are assigned, they may divide the talk or prepare it together with one person presenting. Some coordination with speakers from neighboring weeks may help connect the topics.

Useful Information

Talks. The talks will be held in the **Mathematics & Computer Science (MC)** building at the University of Waterloo, in room **MC 5501**, located on the fifth floor.

Lunches. We are also planning to organize group lunches every other week, alternating between the lounge and the plaza.

Organizers

Leonardo Colò	University of Waterloo
Bruno Sterner	University of Waterloo

Schedule

Fall 2026: QKD and QROM

	Fall 2026		
September 23, 2026	1	**Maher Mamah** **Bruno Sterner**	Introduction to Quantum Computing
September 30, 2026	2	**Elnaz Hessami Pilehrood** **Owen Waldron**	Quantum Information Theory
October 7, 2026	3	**Mojtaba Fadavi** **Taha Hedayat**	Quantum Circuits
October 14, 2026	**Reading Week**		
October 21, 2026	4	**Maggie Simmons**	Quantum Key Distribution
October 28, 2026	5	**Douglas Stebila**	QKD Security and Post-Quantum Cryptography
November 4, 2026	6	**Aodhan Corrigan**	Challenges in Deploying QKD
November 11, 2026	7	**Huanhuan Chen** **Bruce Xu**	Random Oracle Model and Classical Proofs
November 18, 2026	8	**Jonas Janneck** **Camryn Steckel**	QROM Definitions
November 25, 2026	9	**Speaker to be confirmed**	QROM Lemmas
December 2, 2026	10	**Seunghoon Lee**	Example Security Proof in the QROM
December 9, 2026	**Ontario Crypto Days**		

Plan

Introduction to Quantum Computing

Maher Mamah & Bruno Sterner 1

For the first week, we start with a motivation on quantum computing including the basic definitions and discussion on complexity classes. This will serve as a elementary foundation for the rest of the reading group.
References: [7] and [9]

Quantum Information Theory

Elnaz Hessami Pilehrood & Owen Waldron 2

Pure quantum states describe systems whose preparation is known exactly, but realistic quantum information often involves uncertainty, inaccessible subsystems, and interactions with the environment. In this talk, we introduce density operators as a general framework encompassing both pure and mixed states. We then discuss quantum channels, which model physical transformations and common forms of noise affecting quantum systems. Finally, we introduce classical and quantum entropy as measures of uncertainty and information, and explain how these ideas provide basic intuition for quantum cryptographic security.
References: [7] and [9]

Quantum Circuits

Mojtaba Fadavi & Taha Hedayat 3

Equipped with the knowledge of quantum information, we present how to use quantum states to build a quantum algorithm. While classical circuits represent how one makes algorithms using the bits 0 and 1, quantum circuits represent how one makes algorithms using qubits $|\alpha\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$. In this presentation, we will introduce the definition and presentation of quantum gates. Starting with 1-qubit gates and expanding to n-qubit gates. We also present the standard gates such as the Pauli Gates, CNOT, CCNOT, SWAP, etc. Then we proceed to see the effects and interactions of these gates along with some measurements.
References: [7] and [9]

Quantum Key Distribution

Masuda Fatima & Maggie Simmons 4

Quantum Key Distribution (QKD) uses the laws of Quantum Mechanics to allow two parties to produce a shared secret key only known to them without it's security relying on the computational hardness. In this presentation, we will introduce the QKD protocol BB84, and present how it is secure against various attacks. We will also discuss two major phases in BB84: the information reconciliation phase between keys of sender and receiver via error correcting codes and the privacy amplification phase where two communicating parties distill highly secret shared key from a larger body of a shared key, which is only partially secret. Finally, we will give a brief overview of Muckle-based hybrid authenticated key exchange (AKE) protocols.
References: [2], [3], and [5]

QKD Security and Post-Quantum Cryptography in QKD

Douglas Stebila 5

Abstract
References: [3], [5], and [10]

Challenges in Deploying QKD

Aodhan Corrigan 6

Abstract
References: [8] and [11]

The Random Oracle Model and Classical Security Proofs

Huanhuan Chen & Bruce Xu 7

Abstract
References: [6]

Definitions for the Quantum Random Oracle Model

Jonas Janneck & Camryn Steckel 8

Abstract
References: [12] and [13]

QROM Lemmas

Speaker to be confirmed 9

Abstract
References: [1], [4], and [13]

An Example Security Proof in the QROM

Seunghoon Lee 10

Abstract
References: [4]

Week 1

Quantum Computing

Talk by Bruno and Maher.

Classical Computing

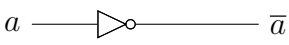
Our modern understanding of computers dates back to the idea of Turing machines. At an informal level a Turing machine consists of 4 main elements:

- Finite state control;
- Tape;
- Read-write tape head; and
- Program.

The finite state control consists of a finite set of internal states $q_1, \cdots, q_m$ including the starting q_s and halting q_h states andacts as the coodinator for the Turing machines operations. The Tape is an infinite one dimensional object consisting of tape squares. Each square contains a symbol drawn from some alphabet (e.g. a bit). The read-write tape head points the position on the tape which is currently readable or writeable. In other words it is the current square being read in the tape. Finally the program is a finite ordered list of program lines of the form $\langle q, x, q', x', s\rangle$. One looks for $\langle q, x, \cdot, \cdot, \cdot\rangle$ such that q is the current state and x is the symbol being read on the tape. If this exists then the program line is executed and the internal state is changed to q and the symbol on the the tape is overwritten by x'. One moves the tape head either left or right or stays in place depending on the value s. If such $\langle q, x, \cdot, \cdot, \cdot\rangle$ does not exist then the internal state is changed to the halting state q_h and the machine halts the operation.

A natural question one can ask is what can we do on Turing machines? This was conclusively answered in the Church-Turing these where in a very broad sense they showed that the class of computable functions that can be made by a Turing machine is equal to the class of functions that can be computed via an algorithm. There are many variants of this statement depending on the specific problem that is being solved but they all have this sort of flavour.

This Turing machine model is a little too idealised and theoretical. Modern computers are finite in size, whereas Turing machines may have unbounded memory. An alternative model for computation that better captures this is the circuit model. This model for computation is equivalent to the Turing machine model in terms of computational power. In a circuit one has wires and gates. These take the role of carrying information around and performing simple computational tasks (respectively). For example, the NOT gate on 1 wire takes a bit and simple flips the bit.



Also, given the equivalence to Turing machines, any circuit within this model comes from an algorithmic description of the procedure.

Complexity Classes

An important question when it comes to problems to be solved in either of these models is computational complexity. Concretely this is the study of time and space resources to solve computational problems on a Turing machine or through a circuit. One initial challenge in capturing this complexity is that different models may require different resources to solve the same problem. For instance a multi-tape Turing machine can solve a problem much quicker than a single-tape machine. The common fix to this is to fix the number of bits, n, as input to the problem and measure resources as bit-operations. An algorithm is said to run in polynomial time if it runs in $O(n^k)$ for some fixed constant k.

As a result there are a broad range of complexity classes which classifies a range of algorithms depending on their computational complexity. The two most well-known complexity classes are P which is the set of problems, with n-bits as input, that can be solved on a Turing machine in polynomial time; and NP which is the set of problems for which you have a candidate solution and you want to check whether that solution is valid in polynomial time. For example, $\text{factoring} \in \text{NP}$ but it is not known whether $\text{factoring} \in \text{P}$. This is encompassed in the well-known P vs NP conjecture. There are many other complexity classes: $\text{PSPACE}, \text{NP-complete}, \text{NP-hard}, \text{BPP}$ and more.

Introduction to Quantum

1. From Classical Bits to Qubits

Classical computation is based on the notion of a *bit*, which represents information using one of two possible states:

$$0 \qquad \text{or} \qquad 1.$$

Mathematically, a classical bit can be viewed as an element of the discrete set $\{0, 1\}$. A classical register containing n bits can therefore store one element from a space of size

$$|\{0, 1\}^n| = 2^n.$$

A quantum computer replaces the classical bit with a *qubit*. Unlike a classical bit, whose state is restricted to two possibilities, a qubit is a vector in a two-dimensional complex Hilbert space:

$$\mathcal{H} = \mathbb{C}^2.$$

The computational basis of this space is given by the orthonormal vectors

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The inner product on this space is defined by

$$\langle \phi | \psi \rangle = \langle \phi | \, | \psi \rangle \,,$$

which determines the geometry of quantum states. The computational basis satisfies

$$\langle 0|0\rangle = \langle 1|1\rangle = 1, \qquad \langle 0|1\rangle = \langle 1|0\rangle = 0.$$

A general pure quantum state is a normalized vector in this Hilbert space:

$$|\psi\rangle = \alpha \, |0\rangle + \beta \, |1\rangle \,,$$

where $\alpha, \beta \in \mathbb{C}$. Normalization requires

$$\langle \psi | \psi \rangle = |\alpha|^2 + |\beta|^2 = 1.$$

The complex numbers α and β are called *probability amplitudes*. Their magnitudes determine measurement probabilities, while their relative phases influence interference effects.

A useful parametrization of a single qubit is

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) |1\rangle \,,$$

where $0 \leq \theta \leq \pi$ and $0 \leq \varphi < 2\pi$. This representation corresponds geometrically to a point on the Bloch sphere.

A physical qubit can be implemented using different quantum systems. A particularly intuitive physical realization of a qubit is provided by the polarization of a single photon. Light is an electromagnetic wave whose electric field oscillates in a direction perpendicular to its propagation direction. When the possible polarization directions are restricted to two orthogonal modes, the photon behaves as a two-level quantum system.

For example, consider horizontal and vertical polarization states. These two states form an orthonormal basis for the polarization Hilbert space:

$$|0\rangle = |H\rangle\,, \qquad |1\rangle = |V\rangle\,.$$

Here, $|H\rangle$ represents a photon whose electric field oscillates horizontally, while $|V\rangle$ represents a photon whose electric field oscillates vertically. Mathematically, these states behave exactly like the computational basis vectors:

$$|H\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |V\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The important quantum feature is that a photon is not restricted to being only horizontally or vertically polarized. It can exist in a coherent superposition of both polarization modes:

$$|\psi\rangle = \alpha\,|H\rangle + \beta\,|V\rangle\,,$$

where

$$|\alpha|^2 + |\beta|^2 = 1.$$

For instance, diagonal polarization corresponds to the state

$$|+\rangle = \frac{|H\rangle + |V\rangle}{\sqrt{2}}.$$

This state does not mean that the photon is half horizontally polarized and half vertically polarized in a classical sense. Rather, the photon exists in a quantum superposition of the two possible polarization modes.

Therefore, the polarization of a single photon provides a direct physical model of a qubit: the two polarization directions define the basis states, while superpositions correspond to arbitrary points on the quantum state space.

2. Measurement of a Qubit

Quantum states evolve according to linear transformations, but measurement produces classical information. The mathematical description of measurement is given by a set of operators satisfying the completeness condition.

For a measurement in the computational basis, the projectors are

$$P_0 = |0\rangle\langle 0|, \qquad P_1 = |1\rangle\langle 1|.$$

The probability of obtaining outcome i is given by the Born rule:

$$P(i) = \langle\psi| P_i |\psi\rangle.$$

For

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

we obtain

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2.$$

After measurement, the state collapses to the corresponding basis state:

$$|\psi\rangle \to |0\rangle \quad \text{or} \quad |1\rangle.$$

For the state

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}},$$

both measurement outcomes occur with equal probability:

$$P(0) = P(1) = \frac{1}{2}.$$

Coming back to the photon realization of a qubit, when measured in the horizontal-vertical basis, the photon is detected as either horizontal or vertical polarization with probabilities

$$P(H) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}, \qquad P(V) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}.$$

The relative phase between polarization components is also physically meaningful. For example,

$$|-\rangle = \frac{|H\rangle - |V\rangle}{\sqrt{2}}$$

has the same measurement probabilities as $|+\rangle$ in the horizontal-vertical basis, but differs in phase. This difference becomes observable when the photon is measured in another basis, illustrating the role of quantum interference.

3. Tensor Products and Multiple Qubits

The state space of multiple quantum systems is constructed using the tensor product. For two qubits,

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B.$$

Since

$$\dim(\mathbb{C}^2) = 2,$$

the dimension of the two-qubit space is

$$\dim(\mathcal{H}_{AB}) = 2^2 = 4.$$

The computational basis becomes

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle.$$

A general two-qubit state is therefore

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle,$$

with

$$\sum_{x\in\{0,1\}^2} |\alpha_x|^2 = 1.$$

For n qubits, the state space is

$$\mathcal{H}_n = (\mathbb{C}^2)^{\otimes n},$$

whose dimension grows exponentially:

$$\dim(\mathcal{H}_n) = 2^n.$$

Therefore, a general quantum state requires 2^n complex amplitudes:

$$|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle .$$

The exponential dimension of the Hilbert space does not mean that a quantum computer directly stores 2^n classical values. Instead, quantum algorithms exploit linear transformations, interference, and entanglement to manipulate the amplitudes in a way that produces computational advantages.

Quantum Computing

We translate this quantum knowledge back to the computing setting. One can define quantum Turing machines as an analogue of the classical Turing machine mentioned at the start (see [Benioff80] for more details on this). More commonly, one uses the quantum circuit model for computation. We will see more of this model in Week 3 but many of the same properties that the classical circuit model has is translated to the quantum analogue.

As a result, we can reason about quantum algorithms as a quantum circuit or a program on a quantum Turing machine. So we can have the same computational complexity discussion as we had before. I.e. measure the amount of resources needed to solve computational problems on these quantum computers given a predescribed amount of input.

There are several complexity classes in this remit, but the most well known is BQP which consists of problems that can be solved in quantum polynomial time on a circuit/Turing machine, where bounded probability error is allowed. One can simulate an classical algorithm as a quantum algorithm, so one clearly has P $\subseteq$ BQP and BQP is the analogy of P in the quantum setting (more precisely BPP is the true analogy here).

Open question: Where does BQP fit within P vs NP? In particular it is unclear where BQP fits within NP where a weird overlap predicted.

Week 2

Quantum Information

Talk by Elnaz and Owen.

Week 3

Quantum Circuts

Talk by Taha and Mojtaba.

Classical Circuits

A classical circuit computes a Boolean function

$$f : \{0,1\}^n \longrightarrow \{0,1\}^m,$$

mapping an n-bit input string to an m-bit output string. The circuit is built by connecting elementary logic gates through wires, where the output of one gate may be used as the input to another. The main gates considered here are the AND, OR, NOT, NAND, and XOR gates. The NOT gate acts on one bit and outputs its complement, while the AND, OR, and XOR gates act on two bits according to

$$\mathrm{AND}(a,b) = a \wedge b, \qquad \mathrm{OR}(a,b) = a \vee b, \qquad \mathrm{NAND}(a,b) = \neg(a \wedge b), \qquad \mathrm{XOR}(a,b) = a \oplus b.$$

By composing these elementary gates, one can construct more complicated Boolean functions. In particular, NAND is functionally complete, meaning that every Boolean function can be implemented using NAND gates alone. Suppose you have a string of n-bits $(b_0, b_1, \ldots, b_{n-1})$, we may

Table 1: Basic classical logic gates.

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$\neg(a \wedge b)$	$a \oplus b$	$a \uparrow b$
0	0	1	0	0	1	0	1
0	1	1	0	1	1	1	1
1	0	0	0	1	1	1	1
1	1	0	1	1	0	0	0

now apply a gate to each one individually or together. This string may represent a number or word or sentence which we wish to edit in some form of way. This can be represented via a visual circuit. For our convention, we will represent the first bit, b_0 at the very top, then work our way down to the n-th bit, b_{n-1} at the bottom. A line is used to represent each bit as time continues onwards. For our convention, the input (i.e. time = 0) will be on the left side of the circuit.

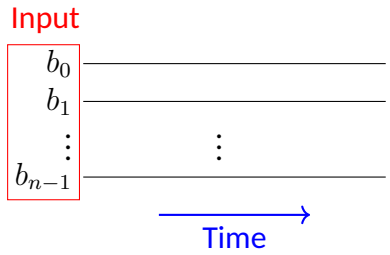
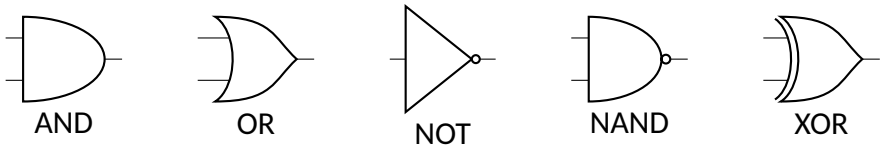


Figure 1: An n-bit classical register $(b_0, b_1, \ldots, b_{n-1})$, with time proceeding from left to right.

The basic classical gates are represented graphically as follows:



Hence if for an input (b_0, b_1, b_2) want the output $((b_0 \wedge b_1) \vee (b_1 \oplus b_2)) \uparrow b_3$, then we could represent this by the circuit

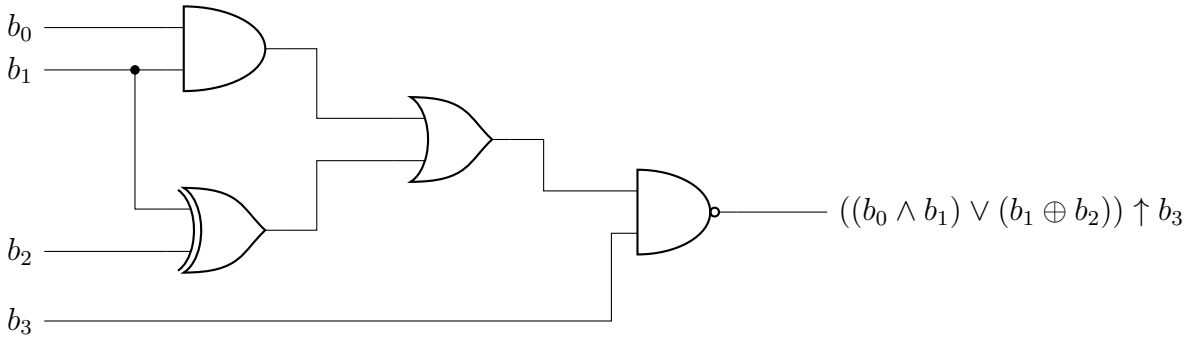


Figure 2: Showing how to draw the circuit which takes (b_0, b_1, b_2, b_3) and maps it to $((b_0 \wedge b_1) \vee (b_1 \oplus b_2)) \uparrow b_3$.

1-Qubit Circuits

A single qubit has computational basis states $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. A general pure qubit state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \qquad |\alpha|^2 + |\beta|^2 = 1.$$

The complex numbers α and β are called *probability amplitudes*. A measurement in the computational basis produces 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. The amplitudes contain both magnitude and phase information. A common phase factor does not change the physical state:

$$|\psi\rangle \sim e^{i\theta}|\psi\rangle.$$

Such a factor is called a *global phase*. In contrast, the relative phase between amplitudes can have observable consequences. For example,

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

Both give a 50–50 distribution when measured directly in the computational basis, but they are different orthogonal quantum states.

Last time, we learned that a qubit need to be a pure state, i.e. of the form $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, but may be a mixed state, represented by a density matrix $\rho = |\psi\rangle\langle\psi|$. More generally, a mixed state is described by

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|, \qquad p_i \geq 0, \qquad \sum_i p_i = 1.$$

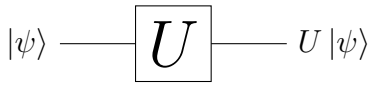


Figure 3: A quantum circtuit starting at the pure state $|\psi\rangle$ having the unitary U applied to it.

Definition. A square matrix ρ is a *density matrix* if $\texttt{Tr}(\rho) = 1$ and $\rho \geq 0$.

In this section we focus primarily on pure states, since quantum gates act on them directly through unitary transformations.

Unitary Transformations

Quantum gates describing closed-system evolution are represented by *unitary matrices*.

Definition. A matrix U is *unitary* if $U^{-1} = U^*$, or equivalently, $U^*U = UU^* = I$, where U^* denotes the conjugate transpose of U.

Hence if we have the pure state qubit $|\psi\rangle$, then after $|\psi\rangle$ has the gate U applied, the output is $U|\psi\rangle$. On the other hand, if we have the mixed state qubit ρ, then after ρ has the gate U applied to it, the output is $U\rho U^*$. Following the same convention as the classical circuits, we have our input on the left and a line represents a qubit (not a bit). A gate applying the unitary U will be a box with U at the center of it to indicate the matrix being applied. See Figure 3.

For a single qubit, a quantum gate is therefore represented by a 2×2 unitary matrix. Unitarity gives several important properties.

1. $\langle U\psi|U\phi\rangle = \langle\psi|U^*U|\phi\rangle = \langle\psi|\phi\rangle$.
2. All eigenvalues satisfy $|\lambda| = 1$.
3. Eigenvectors corresponding to distinct eigenvalues are orthogonal, i.e. U admits an orthonormal eigenbasis.

Thus unitary gates are reversible and preserve the normalization of quantum states.

Pauli Gates Three of the most important single-qubit gates are the Pauli matrices

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Their action on the computational basis is

$$X|0\rangle = |1\rangle, \qquad X|1\rangle = |0\rangle,$$

$$Y|0\rangle = i|1\rangle, \qquad Y|1\rangle = -i|0\rangle,$$
$$Z|0\rangle = |0\rangle, \qquad Z|1\rangle = -|1\rangle.$$

The X gate is the quantum analogue of the classical NOT gate. The Z gate leaves the computational-basis value unchanged but changes the relative phase. The Y gate combines a bit flip with a phase change.

Properties. For every $P \in \{X, Y, Z\}$, the following properties hold:

1. P is both Hermitian and unitary: $P = P^* = P^{-1}$, so P has eigenvalues ± 1 and $P^2 = I$.
2. For $P_1 \neq P_2$, then $P_1 P_2 = -P_2 P_1$.
3. $XY = iZ, \;\; YZ = iX, \quad ZX = iY.$

Other Important Single-Qubit Gates

- **Hadamard gate** The Hadamard gate is

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

  It acts on the computational basis as $H|0\rangle = |+\rangle$, and $H|1\rangle = |-\rangle$, where

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

  Conversely, $H|+\rangle = |0\rangle$, and $H|-\rangle = |1\rangle$. Thus the Hadamard gate changes between the computational basis $\{|0\rangle, |1\rangle\}$ and the Hadamard basis $\{|+\rangle, |-\rangle\}$. It also satisfies $H^2 = I$.

- **Phase gate.** The S gate is

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix},$$

  and acts as

$$S|0\rangle = |0\rangle, \qquad S|1\rangle = i|1\rangle.$$

  It changes the phase of the $|1\rangle$ amplitude by $\pi/2$.

- T **gate.** Another important phase gate is

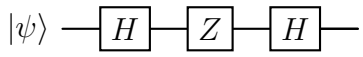
$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

  Its action is

$$T|0\rangle = |0\rangle, \qquad T|1\rangle = e^{i\pi/4}|1\rangle.$$

  Thus the T gate introduces a relative phase of $\pi/4$. Notice that $T^2 = S$.

Composition of Single-Qubit Gates. More complicated circuits are obtained by composing elementary gates. For example,



represents $|\psi\rangle \longmapsto HZH|\psi\rangle$. Since $HZH = X$, the circuit above is equivalent to a single Pauli-X gate.

Superposition, interference, and measurement. A qubit may exist in a superposition $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where the amplitudes can interfere under unitary transformations. For example, $|0\rangle \xrightarrow{H} |+\rangle \xrightarrow{H} |0\rangle$; although measuring $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ would give 0 or 1 with equal probability, the second Hadamard causes constructive and destructive interference and returns the state deterministically to $|0\rangle$. In general, measuring $|\psi\rangle$ in the computational basis gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, after which the state becomes $|0\rangle$ or $|1\rangle$, respectively. Thus, unlike a classical random bit, a quantum superposition retains relative phase information that can affect later transformations and measurements.

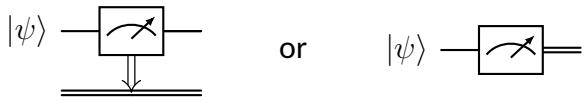
Measurement of the qubit If we are to measure a qubit, we need to decide what we want to keep. Suppose we are going to measure a qubit $|\psi\rangle$ with respect to the computational basis. In such a case, the post measurement state is either

$$\frac{|0\rangle \langle 0|\psi\rangle}{\sqrt{\langle\psi|0\rangle \langle 0|\psi\rangle}} = |0\rangle$$

and the measurement will read "0" with probability $|\langle 0|\psi\rangle|^2$ or the post measurement state is

$$\frac{|1\rangle \langle 1|\psi\rangle}{\sqrt{\langle\psi|1\rangle \langle 1|\psi\rangle}} = |1\rangle$$

and the measurement will read "1" with probability $|\langle 1|\psi\rangle|^2$. Hence, once the measurement is performed, there is a post measurement qubit, and there is a bit outputted. If we want to continue considering our qubit, then we have a line going continuing to the right of the measurement box and a double line (representing the bit) going down. However, if we do not care for the post-measurement state, the measurement box has a double line continuing to the right of the measurement box.



The classical bits will have the same structure as shown before.

n-Qubit Circuits

For n qubits, the state space has dimension 2^n, and an n-qubit gate is represented by a $2^n \times 2^n$ unitary matrix. Single-qubit gates extend naturally to an n-qubit register by tensoring with identity

operators. For example,

$$U \otimes I \qquad \text{or} \qquad U \otimes I \otimes I$$

apply U to one qubit while leaving the others unchanged. More generally, tensor-product gates such as $U_1 \otimes U_2 \otimes \cdots \otimes U_n$ act locally and cannot create entanglement from an initially product state. Multi-qubit gates need not decompose as tensor products of single-qubit operations. Controlled gates such as CNOT and Toffoli can create entanglement for suitable input states. The SWAP gate provides an important distinction: although it cannot be written as $U \otimes V$, it maps product states to product states and therefore does not create entanglement.

2-Qubit Circuits

A two-qubit system has computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, where

$$|xy\rangle = |x\rangle \otimes |y\rangle, \qquad x, y \in \{0, 1\}.$$

Thus a general two-qubit state is

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle, \qquad \sum_{x,y} |\alpha_{xy}|^2 = 1.$$

Since the state space has dimension 4, a two-qubit gate is represented by a 4×4 unitary matrix.

Separable gates and one-qubit gates. Single-qubit gates naturally extend to multi-qubit systems using the tensor product. If U acts on the first qubit and V acts on the second, the corresponding two-qubit operation is $U \otimes V$. In particular, $U \otimes I$ applies U only to the first qubit, while $I \otimes U$ applies it only to the second. Such gates are called *separable* or *local* gates because the two qubits are transformed independently:

$$(U \otimes V)(|\psi\rangle \otimes |\phi\rangle) = U|\psi\rangle \otimes V|\phi\rangle.$$

Therefore, a tensor product of one-qubit gates cannot create entanglement from an initially separable state. For example, applying X to the first qubit and H to the second gives

$$X \otimes H = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \end{pmatrix}.$$

Thus two independent one-qubit operations can be viewed as a single two-qubit unitary transformation.

If we let U, V be unitary 2×2 matrices, and let R be a unitary 4×4 matrix, then we have the following possible gates for a 2-qubit system:



(a) Gate V (b) Gate U (c) Parallel V and U (d) Two-qubit gate R

Figure 4: The four distinct quantum gates

If we let $V = \begin{bmatrix} v_{00} & v_{01} \\ v_{10} & v_{11} \end{bmatrix}$ and $U = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix}$ then Circuit 4a has the associated matrix

$$V \otimes I_2 = \begin{bmatrix} v_{00}I_2 & v_{01}I_2 \\ v_{10}I_2 & v_{11}I_2 \end{bmatrix} = \begin{bmatrix} v_{00} & 0 & v_{01} & 0 \\ 0 & v_{00} & 0 & v_{01} \\ v_{10} & 0 & v_{11} & 0 \\ 0 & v_{10} & 0 & v_{11} \end{bmatrix}$$

On the other hand, Circuit 4b has the associated matrix

$$I_2 \otimes U = \begin{bmatrix} U & 0 \\ 0 & U \end{bmatrix} = \begin{bmatrix} u_{00} & u_{01} & 0 & 0 \\ u_{10} & u_{11} & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}$$

Hence we can see that **order matters a lot**.

Gates 4a, 4b, and 4c are *separable gates*, because they can be represented as a tensor product. Where at Gate 4d need not be a separable gate, it could be an *entangled gate*.

Entangled states. Not every two-qubit state can be written as $|\psi\rangle \otimes |\phi\rangle$. A state that cannot be written in this form is called *entangled*. For example,

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

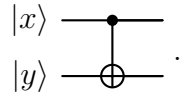
is entangled. Unlike separable operations, genuine two-qubit interactions can create such correlations.

Definition. A quantum gate R is *entangled* if there exists a separable state $|\psi\rangle$ such that $R|\psi\rangle$ is an entangled state.

Controlled-NOT gate. The controlled-NOT gate, also called CNOT or CX, is

$$\mathrm{CNOT} = I \oplus X = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

with circuit representation



Its action on computational-basis states is

$$\mathrm{CNOT}\,|x, y\rangle = |x, y \oplus x\rangle.$$

The first qubit is the *control* and the second is the *target*. If the control is 0, nothing happens; if the control is 1, an X gate is applied to the target. Thus CNOT may be viewed as a *controlled one-qubit operation*: the Pauli-X gate is applied conditionally rather than independently. Unlike $U \otimes V$, CNOT can create entanglement. For example,

$$|00\rangle \xrightarrow{H\otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\mathrm{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

Hence the combination of one-qubit gates and an entangling gate such as CNOT allows behavior that cannot be obtained using one-qubit gates alone.

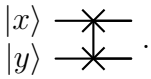
SWAP gate. The SWAP gate exchanges the states of two qubits:

$$\mathrm{SWAP}\,|x, y\rangle = |y, x\rangle,$$

and is represented by

$$\mathrm{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Its circuit symbol is



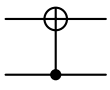
SWAP illustrates an important distinction: it is not a separable gate of the form $U \otimes V$, but it is also *non-entangling*, since

$$|\psi\rangle \otimes |\phi\rangle \longmapsto |\phi\rangle \otimes |\psi\rangle$$

is still a product state. Thus every separable gate is non-entangling, but not every non-entangling two-qubit gate is separable.

In summary, one-qubit gates extend to two-qubit systems as $U \otimes I$, $I \otimes U$, or more generally $U \otimes V$. These local operations act independently and cannot create entanglement. Two-qubit gates such as CNOT introduce interactions between qubits and can create entangled states, while gates such as SWAP act jointly on the two-qubit system without creating entanglement.

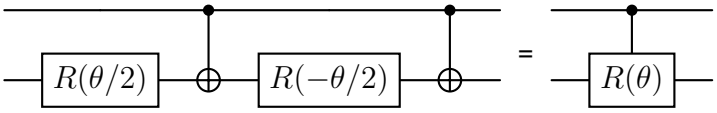
Audience Question 1: What is the matrix representation of



Fun Exercise: Let $R(\theta)$ be the rotation by an angle of θ radians along the X axis. ie. let

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

What is the matrix representation of



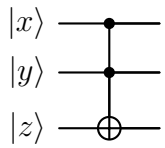
3-Qubit Circuits

A three-qubit system has computational basis

$$\{|000\rangle, |001\rangle, \ldots, |111\rangle\}.$$

A general three-qubit gate is represented by an 8×8 unitary matrix. As with two qubits, single-qubit gates can be applied independently using tensor products. For example, $H \otimes I \otimes I$ applies the Hadamard gate only to the first qubit, while $U \otimes V \otimes W$ applies separate one-qubit gates to the three qubits. Such local operations cannot create entanglement from an initially product state.

Controlled-Controlled-NOT gate. An important three-qubit gate is the *controlled-controlled-NOT* gate, also called CCNOT, CCX, or the Toffoli gate. It has two control qubits and one target qubit:



and acts on computational-basis states as

$$\mathrm{TOFF}\,|x, y, z\rangle = |x, y, z \oplus (x \wedge y)\rangle, \qquad x, y, z \in \{0, 1\}.$$

Thus the target qubit is flipped precisely when both control qubits are equal to 1. Using 2×2 blocks, its matrix can be written compactly as

$$\mathrm{TOFF} = I \oplus I \oplus I \oplus X = \begin{pmatrix} I & 0 & 0 & 0 \\ 0 & I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & X \end{pmatrix}.$$

Equivalently, the gate leaves all computational-basis states unchanged except $|110\rangle \longleftrightarrow |111\rangle$. The Toffoli gate is closely related to the one-qubit Pauli-X gate: the operation on the target is still an X gate, but it is applied only when both controls are 1. In this sense, CNOT is a controlled-X

gate, while Toffoli is a doubly controlled-X gate. When the control qubits are in superposition, such controlled operations can also produce entanglement.

Note that we can generate the Toffoli gate using 5 2-qubit gates. Let V be a unitary operator such that $V^2 = X$. We can construct such a V by taking

$$V = \frac{1}{2}\begin{bmatrix} 1+i & 1-i \\ 1-i & 1+i \end{bmatrix}$$

Now, note that we have

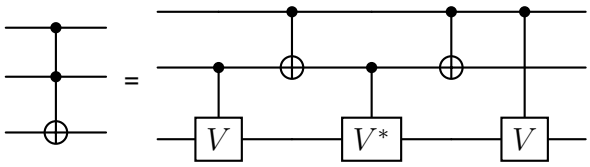


Figure 5: The representation of the Toffoli gate by 5 2-qubit gates.

We can confirm this computation both through the matrix and through the intuition. The matrix representation of the circuit above is

$$(I \otimes (I \oplus V)) \cdot ((I \oplus X) \otimes I) \cdot (I \otimes (I \oplus V^*)) \cdot ((I \oplus X) \otimes I) \cdot ((I \otimes I) \oplus (I \otimes V))$$

Note that this is a large computation. If you put it into SageMath using the following code, then you can verify the computation to be the matrix for the Toffoli gate

```
Iden = Matrix([[1, 0],
               [0, 1]])

X = Matrix([[0, 1],
            [1, 0]])

V = Matrix([[(1+i)/2, (1-i)/2],
            [(1-i)/2, (1+i)/2]])

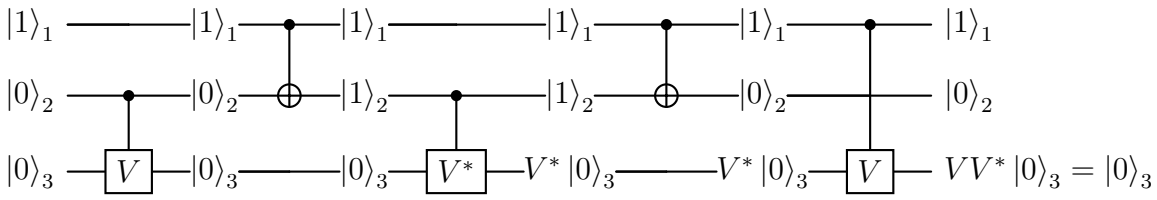
Toff = ((Iden.tensor_product(Iden.block_sum(V)))
        *((Iden.block_sum(X)).tensor_product(Iden))
        *(Iden.tensor_product(Iden.block_sum(V.conjugate_transpose())))
        *((Iden.block_sum(X)).tensor_product(Iden))
        *(Iden.tensor_product(Iden).block_sum(Iden.tensor_product(V))))
```

We can interpreter the equality more generally by considering case by case what should happen for the computational basis.

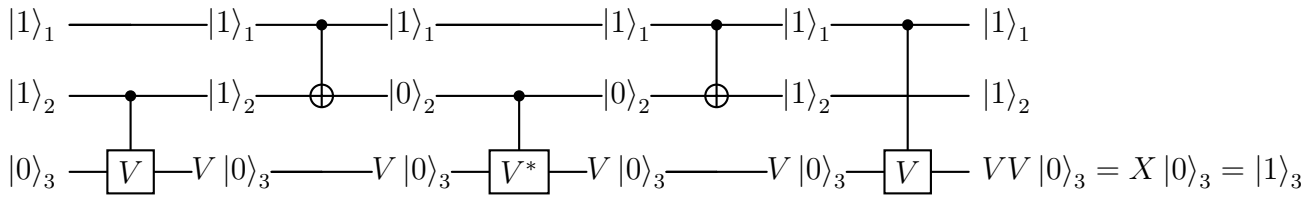
We know that the third qubit should swap between states $|0\rangle$ and $|1\rangle$ if and only if the first and second qubits are in state $|1\rangle$. Otherwise, the third qubit should not change.

Suppose we are in state $|000\rangle = |0\rangle_1 \otimes |0\rangle_2 \otimes |0\rangle_3$ and we apply the 5 2-qubit gates in Figure 5. Since both the first and the second qubit are in state $|0\rangle$ none of the control gates get implemented.

Now, if we are in state $|100\rangle$, then the CNOT gates and the last controlled-V gate get implemented. Hence we have the system



Hence we see that the last state does not change. On the other hand, if we have the state $|110\rangle$, then we have



Hence this circuit is doing precicely what we want with the Toffoli gate. You can probably see how the Toffoli gate represents the classical AND gate but for a quantum setting.

n-Qubit Circuits for $n \geq 4$

A system of n qubits has a basis $\{|x_1 x_2 \cdots x_n\rangle : x_i \in \{0,1\}\}$. Writing $x = x_1 x_2 \cdots x_n \in \{0,1\}^n$, a general pure state is

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \qquad \sum_{x \in \{0,1\}^n} |\alpha_x|^2 = 1.$$

Thus an arbitrary n-qubit state is described by 2^n complex amplitudes, and measuring in the computational basis gives the outcome x with probability $|\alpha_x|^2$.

A gate acting on a n-qubit register is represented by a $2^n \times 2^n$ unitary matrix U, i.e. $U^*U = UU^* = I$. In practice, large circuits are built from gates acting on only one or a few qubits. If a one-qubit gate U acts on the jth qubit, then its action on the full register is

$$I^{\otimes(j-1)} \otimes U \otimes I^{\otimes(n-j)}.$$

More generally, independent one-qubit operations have the form $U_1 \otimes U_2 \otimes \cdots \otimes U_n$.

Product states and entanglement. A n-qubit state is a *product state* if it can be written as

$$|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle.$$

If no such decomposition exists, the state is entangled. For example, starting from $|000\rangle$,

$$|000\rangle \xrightarrow{H\otimes I\otimes I} \frac{|000\rangle + |100\rangle}{\sqrt{2}}$$

produces a superposition but not entanglement. Applying controlled operations can instead correlate the qubits:

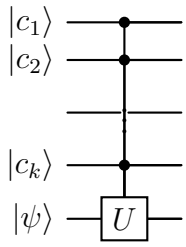
$$\frac{|000\rangle + |100\rangle}{\sqrt{2}} \xrightarrow{\mathrm{CNOT}_{1,2}} \frac{|000\rangle + |110\rangle}{\sqrt{2}} \xrightarrow{\mathrm{CNOT}_{1,3}} \frac{|000\rangle + |111\rangle}{\sqrt{2}}.$$

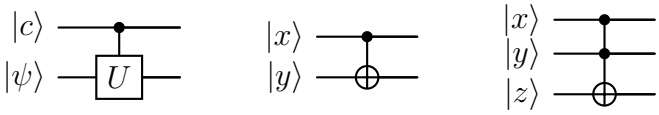
The final state is an example of a three-qubit entangled state.

Controlled operations. The controlled gates introduced earlier extend naturally to larger registers. A controlled-U gate applies U to a target qubit only when the control qubit is $|1\rangle$:

$$|0\rangle|\psi\rangle \longmapsto |0\rangle|\psi\rangle, \qquad |1\rangle|\psi\rangle \longmapsto |1\rangle U|\psi\rangle.$$

For $U = X$, this is the CNOT gate. With two controls, the corresponding controlled-controlled-X operation is the Toffoli gate. More generally, multi-controlled gates apply an operation only when several control conditions are satisfied.





Controlled-U CNOT Toffoli

Composition and circuit complexity. A n-qubit circuit is constructed by composing elementary gates, typically acting on only one or a few qubits:

$$|\psi_{\mathrm{in}}\rangle \xrightarrow{U_1} \xrightarrow{U_2} \cdots \xrightarrow{U_m} |\psi_{\mathrm{out}}\rangle,$$

with overall transformation

$$|\psi_{\mathrm{out}}\rangle = U_m U_{m-1} \cdots U_1 |\psi_{\mathrm{in}}\rangle.$$

Two basic measures of circuit complexity are its *size*, the total number of elementary gates, and its *depth*, the number of sequential layers. Gates acting on disjoint qubits may be executed in parallel, while dependent gates must occur in successive layers. Thus, for example,

$$U_1 \otimes U_2 \otimes \cdots \otimes U_n$$

may be implemented in a single parallel layer.

Universal gate sets. Although a general n-qubit transformation is represented by a $2^n \times 2^n$ unitary matrix, such matrices are not treated as primitive gates. Instead, complex transformations are decomposed into sequences of elementary operations. Suitable single-qubit gates, together with an entangling two-qubit gate such as CNOT, form universal gate sets with which arbitrary quantum computations can be approximated.

We wont prove this result here however, we present it to the intreseted reader:

Theorem. For any $\epsilon > 0$, any quantum circuit can be approximated by X, H, CNOT, and T with less than ϵ error having size $\mathcal{O}(\text{polylog}(\frac{1}{\epsilon}))$.

This collection is called the *Clifford + T* group.

Note that in the above theorem we are using the operator norm to determine the distance between two circuits (represented as single unitaries).

Thus, n-qubit circuits are built by combining local gates, controlled interactions, and measurements. Although the dimension of the state space grows exponentially with n, the circuit itself is described in terms of elementary operations acting on only a small number of qubits at a time. This circuit model underlies larger quantum algorithms such as the quantum Fourier transform, Grover's algorithm, and Shor's algorithm.

Here is the main reason that we actually care about quantum circuits:

Theorem. quantum circuit of size s acting on n qubits can be simulated by a classical circuit of size $\mathcal{O}(sn^2 2^n)$.

proof: it takes 2^n bits to represent a state with n-bit precision, then to do a single gate means to do matrix multiplication costing about n^2 operation. Doing this for s gates, we get $\mathcal{O}(sn^2 2^n)$ many bits operations.

Famous Quantum Circuits

The circuit model becomes particularly useful when studying quantum algorithms. In this section, we consider several fundamental examples that illustrate different sources of quantum advantage. Deutsch's algorithm demonstrates how interference can reveal a global property of a function with fewer oracle queries; Grover's algorithm uses amplitude amplification to accelerate unstructured search; the quantum Fourier transform extracts periodic and phase information; and Shor's algorithm combines these ideas with reversible arithmetic to solve factoring and discrete logarithms efficiently on a quantum computer.

Deutsch's Algorithm

Deutsch's algorithm shows how a quantum circuit can extract information about a function using fewer queries than a deterministic classical algorithm. Consider a Boolean function

$$f : \{0,1\} \longrightarrow \{0,1\}.$$

There are four possible functions:

	$f(0)$	$f(1)$	Type
f_1	0	0	constant
f_2	1	1	constant
f_3	0	1	balanced
f_4	1	0	balanced

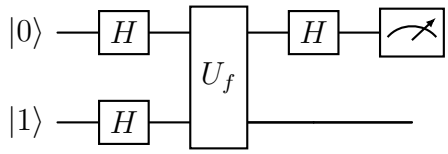
The goal is to determine whether f is *constant*, i.e. $f(0) = f(1)$, or *balanced*, i.e. $f(0) \neq f(1)$.

A deterministic classical algorithm must evaluate both $f(0)$ and $f(1)$ in the worst case. Deutsch's quantum algorithm determines the answer using only one evaluation of the corresponding quantum oracle.

Quantum oracle. The function f is represented by a reversible unitary operation U_f defined by

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle,$$

where $x, y \in \{0,1\}$. The second qubit is used to make the operation reversible. Deutsch's circuit is



Starting from $|0\rangle|1\rangle$, the Hadamard layer prepares

$$(H \otimes H)|0,1\rangle \longmapsto \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |+\rangle|-\rangle.$$

Phase kickback. Since $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$, we have

$$U_f\big(|x\rangle|-\rangle\big) = \frac{1}{\sqrt{2}}\left(U_f|x,0\rangle - U_f|x,1\rangle\right) = \frac{1}{\sqrt{2}}\left(|x, 0 \oplus f(x)\rangle - |x, 1 \oplus f(x)\rangle\right) = (-1)^{f(x)}|x\rangle|-\rangle.$$

Hence

$$U_f(|+\rangle|-\rangle) = \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \otimes |-\rangle.$$

Applying H to the first qubit gives

$$\frac{1}{2}\left[\big((-1)^{f(0)} + (-1)^{f(1)}\big)|0\rangle + \big((-1)^{f(0)} - (-1)^{f(1)}\big)|1\rangle\right].$$

Therefore, if measurement is 0, f is constant; otherwise f is balanced.

Deutsch's algorithm does not obtain the individual values $f(0)$ and $f(1)$. Instead, the oracle encodes them into relative phases, and the final Hadamard gate causes these amplitudes to interfere. Constructive and destructive interference reveal the global property $f(0) \oplus f(1)$ without separately determining the two function values.

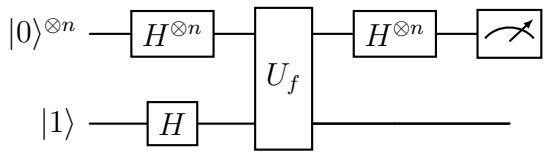
Deutsch–Jozsa Algorithm. Deutsch's algorithm generalizes naturally to the *Deutsch–Jozsa algorithm*. Consider a Boolean function

$$f : \{0,1\}^n \longrightarrow \{0,1\},$$

with the promise that f is either *constant*, meaning that $f(x)$ is the same for every x, or *balanced*, meaning that

$$|\{x : f(x) = 0\}| = |\{x : f(x) = 1\}| = 2^{n-1}.$$

The goal is to determine whether f is constant or balanced. We do not develop the full derivation here; using the Deutsch–Jozsa circuit, a measurement of the first register gives 0^n then f is constant, while any nonzero outcome implies that f is balanced.



For a deterministic classical algorithm, in the worst case one must query $2^{n-1}+1$ inputs. While, the Deutsch–Jozsa algorithm requires only one oracle query. Thus Deutsch–Jozsa gives an exponential separation in exact deterministic query complexity.

Grover's Algorithm

Grover's algorithm searches an unstructured space with a quadratic improvement in query complexity. It is a fundamental example of *amplitude amplification*, where quantum interference is used to increase the amplitudes of desired states and decrease those of undesired states. Let

$$f : \{0,1\}^n \longrightarrow \{0,1\},$$

and suppose that we want to find an input x such that $f(x) = 1$. Writing $N = 2^n$, let

$$A_1 = \{x : f(x) = 1\}, \qquad A_0 = \{x : f(x) = 0\},$$

and suppose that there are $M = |A_1|$ satisfying inputs. Classically, finding a satisfying input requires $\mathcal{O}(N/M)$ queries in general, whereas Grover's algorithm uses

$$\mathcal{O}\left(\sqrt{\frac{N}{M}}\right)$$

quantum queries.

Initialization. Starting from $|0^n\rangle$, Hadamard gates prepare the uniform superposition

$$|s\rangle = H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{N}} \sum_{x\in\{0,1\}^n} |x\rangle.$$

To understand the algorithm, define the normalized superpositions of the satisfying and unsatisfying inputs:

$$|A_1\rangle = \frac{1}{\sqrt{M}} \sum_{x\in A_1} |x\rangle, \qquad |A_0\rangle = \frac{1}{\sqrt{N-M}} \sum_{x\in A_0} |x\rangle.$$

These states are orthogonal, and the initial state can be written as

$$|s\rangle = \sqrt{\frac{N-M}{N}}\,|A_0\rangle + \sqrt{\frac{M}{N}}\,|A_1\rangle.$$

Let θ satisfy

$$\cos\theta = \sqrt{\frac{N-M}{N}}, \qquad \sin\theta = \sqrt{\frac{M}{N}}.$$

Then

$$|s\rangle = \cos\theta\,|A_0\rangle + \sin\theta\,|A_1\rangle.$$

Thus Grover's algorithm can be analyzed entirely in the two-dimensional space spanned by $|A_0\rangle$ and $|A_1\rangle$.

Oracle reflection. The standard reversible oracle is

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle.$$

As in Deutsch's algorithm, preparing the second register in $|-\rangle$ gives phase kickback:

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle.$$

Ignoring the unchanged $|-\rangle$ register, we may therefore write the phase oracle as

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle \implies \quad O_f|A_0\rangle = |A_0\rangle, \quad O_f|A_1\rangle = -|A_1\rangle.$$

Hence O_f is a reflection: it keeps the unsatisfying component fixed and reverses the sign of the satisfying component.

Diffusion reflection. The second operation in Grover's algorithm is the diffusion operator

$$D = 2|s\rangle\langle s| - I,$$

where

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle \xrightarrow{|\psi\rangle=\sum_x \alpha_x|x\rangle,} \langle s|\psi\rangle = \frac{1}{\sqrt{N}} \sum_x \alpha_x = \sqrt{N}\,\overline{\alpha}, \qquad \text{where } \overline{\alpha} = \frac{1}{N}\sum_x \alpha_x.$$

Therefore,

$$|s\rangle\langle s|\psi\rangle = \overline{\alpha}\sum_x |x\rangle,$$

and hence

$$D|\psi\rangle = (2|s\rangle\langle s| - I)|\psi\rangle = \sum_x (2\overline{\alpha} - \alpha_x)|x\rangle.$$

Thus each amplitude is reflected about the average:

$$\alpha_x \longmapsto 2\overline{\alpha} - \alpha_x.$$

Remark. *To see this geometrically, write* $\alpha_x = \overline{\alpha} + (\alpha_x - \overline{\alpha})$. *Hence*

$$|\psi\rangle = \underbrace{\overline{\alpha}\sum_x |x\rangle}_{\text{parallel to } |s\rangle} + \underbrace{\sum_x (\alpha_x - \overline{\alpha})|x\rangle}_{\text{orthogonal to } |s\rangle}.$$ [1]

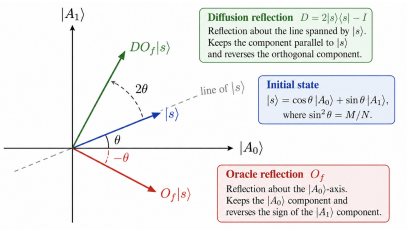
The diffusion operator keeps the component parallel to $|s\rangle$ *and reverses the orthogonal component. Therefore,*

$$D|\psi\rangle = \overline{\alpha}\sum_x |x\rangle - \sum_x (\alpha_x - \overline{\alpha})|x\rangle.$$

Thus the deviation from the average changes sign:

$$\alpha_x - \overline{\alpha} \longmapsto -(\alpha_x - \overline{\alpha}),$$

which explains the name inversion about the average.



Let

$$O_0|x\rangle = (-1)^{[x=0^n]}|x\rangle,$$

where $[x = 0^n]$ is 1 if $x = 0^n$ and 0 otherwise. Thus O_0 flips the phase of $|0^n\rangle$ and leaves every other computational-basis state unchanged. Hence

$$O_0 = I - 2|0^n\rangle\langle 0^n|.$$ [2]

---

[1] Indeed, $\overline{\alpha}\sum_x |x\rangle = \sqrt{N}\,\overline{\alpha}\,|s\rangle$, while $\frac{1}{\sqrt{N}}\sum_x(\alpha_x - \overline{\alpha}) = 0$, so the second term is orthogonal to $|s\rangle$.

[2] Recall that

$$\langle 0^n|x\rangle = \prod_{i=1}^n \delta_{0,x_i}, \qquad |x\rangle = |x_1\rangle \otimes \cdots \otimes |x_n\rangle.$$

Hence $\langle 0^n|0^n\rangle = 1$, while $\langle 0^n|x\rangle = 0$ for $x \neq 0^n$. Therefore,

$$(I - 2|0^n\rangle\langle 0^n|)|0^n\rangle = -|0^n\rangle, \qquad (I - 2|0^n\rangle\langle 0^n|)|x\rangle = |x\rangle \quad (x \neq 0^n).$$

Since

$$|s\rangle = H^{\otimes n}|0^n\rangle,$$

we obtain

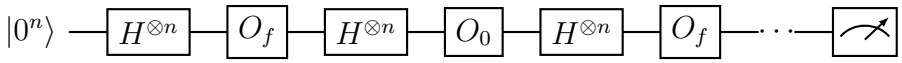
$$-D = I - 2|s\rangle\langle s| = H^{\otimes n} O_0 H^{\otimes n} \Longrightarrow \boxed{D = -H^{\otimes n} O_0 H^{\otimes n},}$$

where the minus sign is only a global phase.

Grover iteration as a rotation. One Grover iteration is $G = DO_f$. Since O_f and D are both reflections, their composition is a rotation. The angle between their reflection axes is θ, so each Grover iteration rotates the state by 2θ toward the satisfying state $|A_1\rangle$. The iteration can equivalently be written as

$$G = -H^{\otimes n} O_0 H^{\otimes n} O_f.$$

A high-level circuit is therefore



where the sequence

$$O_f \longrightarrow H^{\otimes n} \longrightarrow O_0 \longrightarrow H^{\otimes n}$$

is repeated. After k Grover iterations,

$$G^k|s\rangle = \cos((2k+1)\theta)|A_0\rangle + \sin((2k+1)\theta)|A_1\rangle.$$

Therefore, the probability of measuring a satisfying input is

$$P_k = \sin^2((2k+1)\theta).$$

This formula explains the algorithm: each iteration rotates amplitude from the unsatisfying subspace toward the satisfying subspace.

Number of iterations. We want the state to be as close as possible to $|A_1\rangle$, so we choose k such that

$$(2k+1)\theta \approx \frac{\pi}{2}.$$

Hence

$$k \approx \frac{\pi}{4\theta} - \frac{1}{2}.$$

When $M \ll N$,

$$\sin\theta = \sqrt{\frac{M}{N}} \qquad \Longrightarrow \qquad \theta \approx \sqrt{\frac{M}{N}},$$

and therefore

$$k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}.$$

For a unique satisfying input, $M = 1$, so

$$k \approx \frac{\pi}{4}\sqrt{N}.$$

After this many iterations, measurement produces the marked input with high probability.

It is important not to continue the iterations indefinitely: since Grover's operator is a rotation, too many iterations rotate the state past $|A_1\rangle$ and the success probability begins to decrease.

Cryptographic significance. Grover's algorithm is particularly relevant to brute-force attacks on symmetric cryptography. A classical exhaustive search over a k-bit key space requires approximately 2^k key trials, while an idealized Grover search requires approximately $2^{k/2}$ quantum oracle evaluations. Similarly, a generic preimage search for an n-bit hash value is reduced from order 2^n classical evaluations to order $2^{n/2}$ quantum evaluations.

Quantum Fourier Transform

The classical discrete Fourier transform (DFT) maps a vector $(x_0, \ldots, x_{N-1}) \to (y_0, \ldots, y_{N-1})$, where $x_i, y_i \in \mathbb{C}^N$ and

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k/N}.$$

A direct computation requires $\mathcal{O}(N^2)$ operations, while the fast Fourier transform (FFT) reduces this to $\mathcal{O}(N \log N)$. For an n-qubit system, however, $N = 2^n$, so even the FFT complexity becomes $\mathcal{O}(N \log N) = \mathcal{O}(n2^n)$, which is exponential in the number of qubits n.

The natural quantum question is therefore: can the corresponding Fourier transformation on a quantum state be implemented using only polynomially many quantum gates? Remarkably, the answer is yes. The *quantum Fourier transform* (QFT) can be implemented using $\mathcal{O}(n^2) = \mathcal{O}(\log N)^2)$ elementary quantum gates. The QFT is particularly useful for extracting phase and periodic information and is a central component of Shor's algorithm.

Let

$$N = 2^n, \qquad \omega = e^{2\pi i/N}.$$

The QFT is the unitary transformation

$$F_N = \frac{1}{\sqrt{N}} \left[\omega^{xy}\right]_{x,y=0}^{N-1},$$

or equivalently,

$$F_N|x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega^{xy}|y\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i x y/N}|y\rangle.$$

The roots of unity satisfy

$$\sum_{y=0}^{N-1} \omega^{ky} = \begin{cases} N, & k \equiv 0 \pmod N, \\ 0, & k \not\equiv 0 \pmod N. \end{cases}$$

Hence

$$\langle x'|F_N^* F_N|x\rangle = \frac{1}{N}\sum_{y=0}^{N-1}\omega^{y(x-x')} = \delta_{x,x'},$$

so F_N is unitary.[3]

Action on amplitudes. For a general state

$$|\psi\rangle = \sum_{x=0}^{N-1}\alpha_x|x\rangle,$$

linearity gives

$$F_N|\psi\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}\alpha_x e^{2\pi i xy/N}|y\rangle = \sum_{y=0}^{N-1}\widetilde{\alpha}_y|y\rangle,$$

where $\widetilde{\alpha}_y = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}\alpha_x e^{2\pi i xy/N}$. Thus the QFT performs a discrete Fourier transform on the quantum amplitudes.

For example,

$$F_N|0\rangle = \frac{1}{\sqrt{N}}\sum_{y=0}^{N-1}|y\rangle, \xrightarrow{N=2} F_2 = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix} = H.$$

Thus the Hadamard gate is precisely the one-qubit Fourier transform.

QFT on n qubits. Write

$$y = y_1 2^{n-1} + y_2 2^{n-2} + \cdots + y_n, \qquad y_k \in \{0,1\}.$$

Then

$$\frac{xy}{2^n} = \sum_{k=1}^{n}\frac{xy_k}{2^k},$$

and therefore

$$F_N|x\rangle = \frac{1}{\sqrt{2^n}}\sum_{y_1,\ldots,y_n}\prod_{k=1}^{n}e^{2\pi i x y_k/2^k}|y_1\cdots y_n\rangle = \frac{1}{\sqrt{2^n}}\bigotimes_{k=1}^{n}\left(|0\rangle + e^{2\pi i x/2^k}|1\rangle\right).$$

Thus the Fourier basis state can be constructed one qubit at a time.

If $x = x_1x_2\cdots x_n$ is the binary representation of x, with x_1 the most significant bit, then

$$e^{2\pi i x/2^k} = e^{2\pi i(0.x_{n-k+1}\cdots x_n)},$$

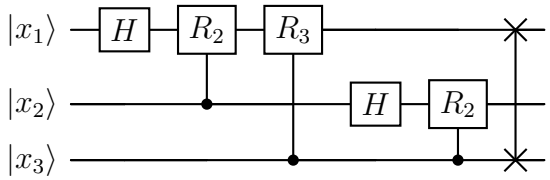
where $0.x_{n-k+1}\cdots x_n$ denotes a binary fraction. **Hence the QFT encodes the bits of x into relative phases.**

[3] $\langle x'|F_N^* F_N|x\rangle = \frac{1}{N}\sum_{y=0}^{N-1}\omega^{y(x-x')}$. If $x = x'$, the sum equals N; otherwise 0. since $\omega^N = 1$. Hence $\langle x'|F_N^* F_N|x\rangle = \delta_{x,x'}$. Therefore, $F_N^* F_N = I$, i.e. F_N is unitary.

QFT circuit. These phases are generated using Hadamard gates and controlled phase rotations

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}.$$

For example, the three-qubit QFT can be implemented as



The Hadamard gates create superpositions, while the controlled R_k gates introduce the required relative phases. The output qubits appear in reverse order, so SWAP gates are used at the end to reverse their order.

Inverse QFT. Since F_N is unitary, we have $F_N^{-1} = F_N^*$. Also,

$$F_N^{-1}|y\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} e^{-2\pi i xy/N}|x\rangle.$$

At the circuit level, the inverse QFT is obtained by reversing the order of the QFT gates and replacing every R_k by R_k^{-1}.

Remark. *In Shor's algorithm, this inverse QFT is used to extract information about the period of the modular function*

$$f(x) = a^x \bmod N,$$

thereby reducing factoring to quantum order finding.

Why does the QFT reveal periodicity? Suppose that $r \mid N$ and consider the normalized periodic state

$$|\psi_r\rangle = \sqrt{\frac{r}{N}} \sum_{j=0}^{N/r-1} |x_0 + jr\rangle,$$

whose populated basis states are separated by the period r. Applying the QFT,

$$F_N|\psi_r\rangle = \sqrt{\frac{r}{N}} \sum_{j=0}^{N/r-1} F_N|x_0 + jr\rangle = \sqrt{\frac{r}{N}} \sum_{j=0}^{N/r-1} \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i(x_0+jr)y/N}|y\rangle = \frac{\sqrt{r}}{N} \sum_{y=0}^{N-1} e^{2\pi i x_0 y/N} \left(\sum_{j=0}^{N/r-1} e^{2\pi i jry/N} \right)$$

Thus the amplitude associated with $|y\rangle$ depends on the sum

$$S_y = \sum_{j=0}^{N/r-1} e^{2\pi i jry/N} = \begin{cases} N/r, & e^{2\pi iry/N} = 1, \\ 0, & e^{2\pi iry/N} \neq 1. \end{cases}$$

Therefore, destructive interference eliminates all values of y except those satisfying

$$e^{2\pi iry/N} = 1 \quad \Longleftrightarrow \quad ry \equiv 0 \pmod{N}.$$

Since $r \mid N$, the surviving values are $y = \ell\frac{N}{r}$, for $\ell = 0, \ldots, r-1$. Consequently,

$$F_N|\psi_r\rangle = \frac{1}{\sqrt{r}} \sum_{\ell=0}^{r-1} e^{2\pi i x_0 \ell/r} \left|\ell\frac{N}{r}\right\rangle.$$

Thus measurement outputs $|\ell\frac{N}{r}\rangle$, so dividing by N we will have $\frac{\ell}{r}$, so the QFT converts the period r into measurable frequency information. When $r \nmid N$, the probabilities are instead concentrated near values satisfying

$$\frac{y}{N} \approx \frac{\ell}{r}.$$

Circuit complexity. The exact QFT on n qubits uses n Hadamard gates, $\frac{n(n-1)}{2}$ controlled phase rotations, and at most $\left\lfloor \frac{n}{2} \right\rfloor$ SWAP gates. Hence

$$\text{size}(F_{2^n}) = \mathcal{O}(n^2) = \mathcal{O}((\log N)^2).$$

Small-angle rotations may be omitted to obtain an approximate QFT with fewer gates.

Remark. *It is important that the QFT does not allow us to read all 2^n Fourier coefficients: a measurement produces only one classical outcome. Its usefulness comes from algorithms in which global information, such as a period or phase, is concentrated into measurement outcomes.*

Shor's Algorithm

Shor's algorithm gives a polynomial-time quantum algorithm for integer factorization by reducing factoring to *order finding*. Let m be the composite integer to be factored, and choose

$$a \in \{2, \ldots, m-1\}.$$

If $\gcd(a, m) > 1$, then a nontrivial factor of m has already been found. Otherwise, a is invertible modulo m.

The *order* of a modulo m is the smallest positive integer r such that

$$a^r \equiv 1 \pmod{m}.$$

Equivalently, the modular function

$$f(x) = a^x \bmod m$$

is periodic with period r, since

$$f(x+r) = a^{x+r} \equiv a^x a^r \equiv a^x \pmod{m}.$$

The quantum part of Shor's algorithm creates a state containing this periodic structure and uses the inverse QFT to convert it into measurable frequency information, from which r can be recovered by classical post-processing.

Knowing r can reveal factors of m. If r is even, then

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{m}.$$

Hence, provided

$$a^{r/2} \not\equiv -1 \pmod{m},$$

the quantities

$$\gcd(a^{r/2} - 1, m), \qquad \gcd(a^{r/2} + 1, m)$$

give nontrivial factors of m. If these conditions fail, another value of a is chosen.

Thus the main computational task is

$$\boxed{\text{find the order } r \text{ of } a \pmod{m}.}$$

Quantum order finding. Let $n = \lceil \log_2 m \rceil$, and choose

$$Q = 2^t, \qquad t \approx 2n.$$

Starting from

$$|0\rangle^{\otimes t}|1\rangle,$$

Hadamard gates on the first register create

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|1\rangle.$$

We then compute modular exponentiation reversibly. Let $U_{\exp}$ be the unitary operation satisfying

$$U_{\exp}\big(|x\rangle|1\rangle\big) = |x\rangle|a^x \bmod m\rangle.$$

Thus the first register stores x, while the second register coherently computes the corresponding value $a^x \bmod m$.

By linearity,

$$U_{\exp}\left(\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|1\rangle\right) = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|a^x \bmod m\rangle.$$

Since $a^{x+r} \equiv a^x \pmod{m}$, the values in the second register repeat with period r. For example, if measuring the second register gives some value $a^{x_0} \bmod m$, then the first register collapses approximately to

$$|\phi_{x_0}\rangle = \frac{1}{\sqrt{L}} \sum_{j=0}^{L-1} |x_0 + jr\rangle, \qquad L \approx \frac{Q}{r}.$$

Thus the first register has exactly the periodic structure considered in the previous QFT subsection.

Extracting the period with the QFT. Applying the inverse QFT to this periodic state gives

$$F_Q^{-1}|\phi_{x_0}\rangle = \frac{1}{\sqrt{L}}\sum_{j=0}^{L-1} F_Q^{-1}|x_0 + jr\rangle = \frac{1}{\sqrt{LQ}}\sum_{y=0}^{Q-1} e^{-2\pi i x_0 y/Q}\left(\sum_{j=0}^{L-1} e^{-2\pi i j r y/Q}\right)|y\rangle.$$

As in the previous subsection, the inner geometric sum exhibits constructive interference when

$$\frac{ry}{Q}$$

is close to an integer. Therefore the measurement probabilities are concentrated near values satisfying

$$\boxed{\frac{y}{Q} \approx \frac{k}{r}}$$

for some $k \in \{0, \ldots, r-1\}$. Thus the quantum computation converts the unknown period r into a measurable approximation to a rational number whose denominator is related to r.

Classical post-processing. After measuring y, we know the rational number $\frac{y}{Q}$, and the quantum computation tells us that

$$\frac{y}{Q} \approx \frac{k}{r}.$$

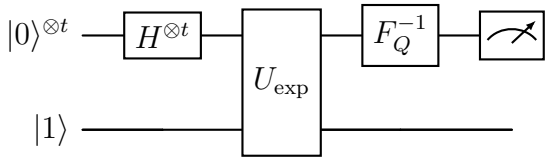
Since $r < m$ and Q is chosen of order m^2, this approximation is sufficiently accurate for the classical *continued-fractions algorithm* to recover a candidate denominator r.

The candidate is checked efficiently by verifying

$$a^r \equiv 1 \pmod{m}.$$

If necessary, the quantum experiment is repeated; different measurements may give different fractions k/r, from which the order can be recovered.

The quantum part of Shor's factoring algorithm can therefore be represented as



Hence the main quantum computation is

superposition $\longrightarrow$ modular exponentiation $\longrightarrow$ periodic state $\longrightarrow$ F_Q^{-1} $\longrightarrow$ measurement.

Complexity and discrete logarithms. Let $n = \lceil \log_2 m \rceil$. Writing the exponent in binary as

$$x = \sum_{j=0}^{t-1} x_j 2^j, \qquad t = O(n),$$

we have

$$a^x = \prod_{j=0}^{t-1} \left(a^{2^j}\right)^{x_j}.$$

Thus repeated squaring implements modular exponentiation using one controlled modular multiplication for each exponent bit, for a total of $O(n)$ controlled modular multiplications. With straightforward reversible arithmetic, each n-bit modular multiplication costs $O(n^2)$ elementary gates, giving

$$O(n^3)$$

gates for modular exponentiation. The inverse QFT acts on $\mathcal{O}(n)$ qubits and requires only $\mathcal{O}(n^2)$ gates, so reversible modular arithmetic dominates. Thus, in a standard textbook implementation,

$$\boxed{T_{\text{factor}}(n) = \mathcal{O}(n^3),}$$

up to lower-order and implementation-dependent factors.

Shor's Algorithm for Discret Logarithm Problem. Shor's method also applies to the discrete logarithm problem. If $G = \langle g \rangle$ and

$$h = g^s,$$

then the goal is to recover s. Consider

$$f(x, y) = g^x h^{-y} = g^{x-sy}.$$

This function has a two-dimensional periodic structure; for example,

$$f(x + s, y + 1) = f(x, y).$$

A quantum superposition over (x, y) together with a two-dimensional QFT reveals this hidden periodicity, from which s can be recovered by classical post-processing.

For discrete logarithms in groups with efficiently implementable group operations, Shor's algorithm likewise runs in polynomial time. For standard n-bit arithmetic implementations, one obtains the textbook estimate

$$\boxed{T_{\text{DLP}}(n) = \mathcal{O}(n^3),}$$

again up to implementation-dependent factors.

Week 4

Quantum Key Distribution

Talk by .

Week 5

Quantum Security

Talk by .

Week 6

Deploying QKD

Invited talk by .

Week 7

Classical Random Oracle Model

Talk by .

Week 8

Quantum Random Oracle Model

Talk by .

Week 9

QROM Lemmas

Talk by .

Week 10

QROM Security Proof

Talk by .

References

Bibliography:

[1] A. Ambainis, M. Hamburg, and D. Unruh. Quantum security proofs using semi-classical oracles. Cryptology ePrint Archive, 2018/904. https://eprint.iacr.org/2018/904.

[2] C. H. Bennett and G. Brassard. Quantum cryptography: Public key distribution and coin tossing. 1984; reprinted in *Theoretical Computer Science* 560, pp. 7–11, 2014. https://arxiv.org/abs/2003.06557.

[3] S. Bruckner, S. Ramacher, and C. Striecks. Muckle+: End-to-End Hybrid Authenticated Key Exchanges. *PQCrypto*, 2023. https://eprint.iacr.org/2023/653.

[4] J. Don, S. Fehr, C. Majenz, and C. Schaffner. Security of the Fiat–Shamir Transformation in the Quantum Random-Oracle Model. 2019. https://arxiv.org/abs/1902.07556.

[5] B. Dowling, T. B. Hansen, and K. G. Paterson. Many a Mickle Makes a Muckle: A Framework for Provably Quantum-Secure Hybrid Key Exchange. *PQCrypto*, 2020. https://eprint.iacr.org/2020/099.

[6] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. See the random-oracle model and proof techniques.

[7] D. Leung. *Introduction to Quantum Information Processing*, University of Waterloo course materials, Winter 2025. https://www.math.uwaterloo.ca/~wcleung/intro-qinfo-w25.html.

[8] H.-K. Lo, M. Curty, and K. Tamaki. Secure Quantum Key Distribution. 2015. https://arxiv.org/abs/1505.05303.

[9] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[10] M. Tomamichel and A. Leverrier. A largely self-contained and complete security proof for quantum key distribution. 2015. https://arxiv.org/abs/1506.08458.

[11] V. Zapatero, Á. Navarrete, and M. Curty. Implementation security in quantum key distribution. 2023. https://arxiv.org/abs/2310.20377.

[12] M. Zhandry. How to Construct Quantum Random Functions. *FOCS*, 2012. https://eprint.iacr.org/2012/182.

[13] M. Zhandry. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. 2018. https://eprint.iacr.org/2018/276.